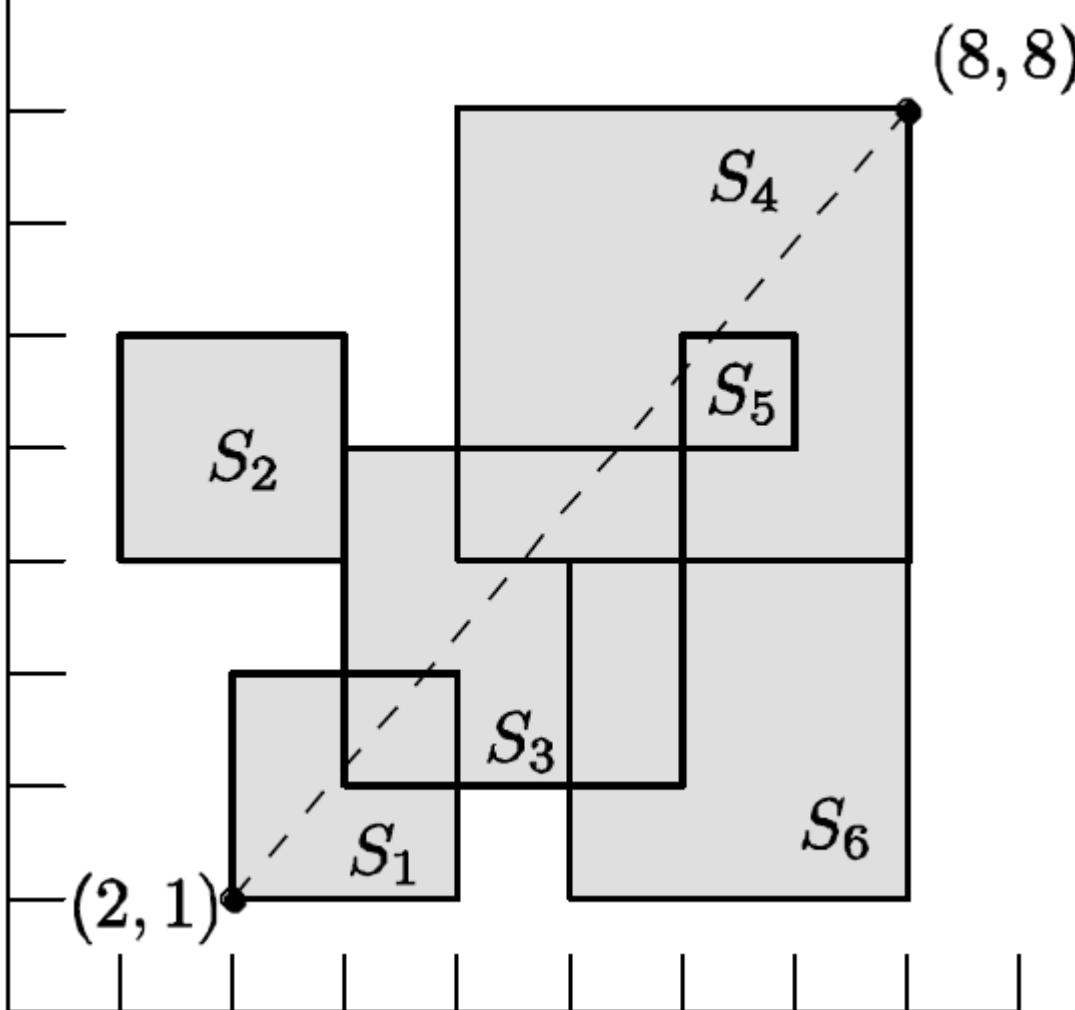


题目链接

本题是2009年ICPC亚洲区域赛首尔赛区的F题

题意

给定平面上n个边平行于坐标轴的矩形，在它们的顶点中找出两个欧几里得距离最大的点。如下图所示，距离最大的是S1的左下角和S4的右上角。正方形可以重合或者交叉。
你的任务是输出这个最大距离的平方。



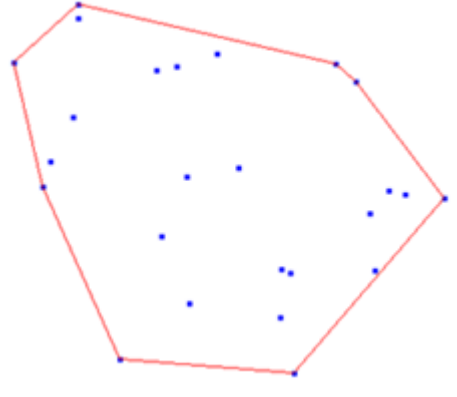
分析

首先把矩形的所有顶点求出，得到一个点集，则本题的目标就是要求出这个点集的直径（diameter），即点之间的最大距离。

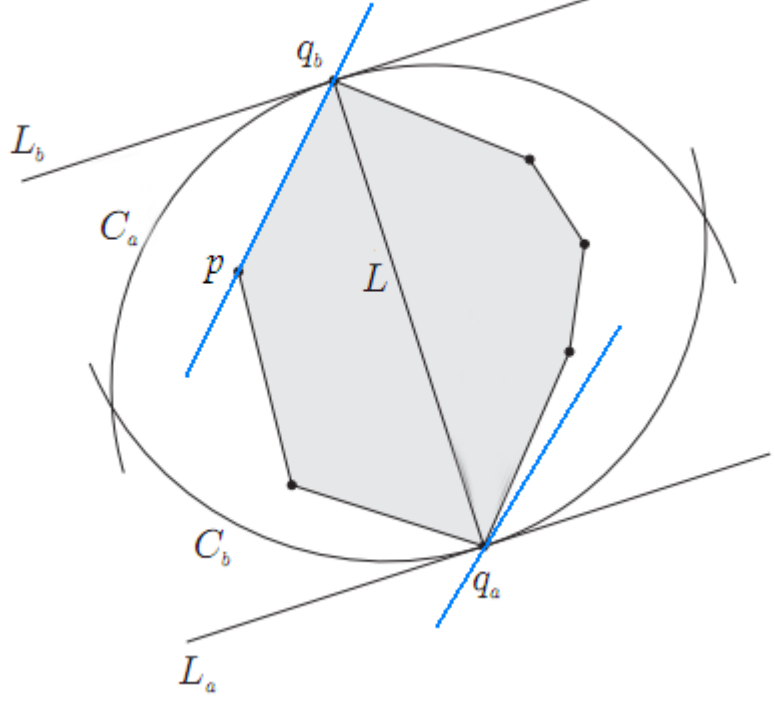
这是经典问题：求最远点对。与之对应的另外一个经典问题是最临近点对。

“寻找最近点对”是用分治策略降低复杂度（并且利用抽屉原理进行分析得出一个跟有趣的结论将时间复杂度优化到 $O(n \log n)$ ），而“寻找最远点对”可利用几何性质将时间复杂度优化到 $O(n \log n)$ 。

注意到对于平面上的n个点，最远点对必然存在于这n个点所构成的一个凸包上（求点集的直径变成了求凸包的直径），那么可以排除大量点，如下图所示：



找最远点对的枚举方法是旋转卡壳（rotating calipers）法：可以想象有两条平行线，“卡”住这个凸包，然后卡紧的情况下旋转一圈，肯定就能找到凸包直径，也就找到了最远点对。



细说一下旋转卡壳法：如果qa, qb是凸包上最远两点，必然可以分别过qa, qb画出一对平行线。通过旋转这对平行线，我们可以让它和凸包上的一条边重合，如图中蓝色直线，可以注意到，qa是凸包上离p和qb所在直线最远的点。逆时针枚举凸包上的所有边，对每一条边找出凸包上离该边最远的顶点，计算这个顶点到该边两个端点的距离，并记录最大的值。注意到当逆时针枚举边的时候，最远点的变化也是逆时针的，这样就可以不用从头寻找最远点，而是紧接着上一次的最远点继续寻找，于是得到了O(n)的算法。

Andrew算法求完“下凸包”时记录下最后一个顶点（即最右的顶点）的索引k，则用k-1作为将来旋转卡壳法枚举凸包首条边时最远顶点即可（实际上首条边的最远顶点≥k-1）。

AC代码

```
#include <iostream>
#include <algorithm>
using namespace std;

struct Point {
    int x, y;
    Point(int x = 0, int y = 0): x(x), y(y) {}
};
typedef Point Vector;

Vector operator- (const Point& A, const Point& B) {
    return Vector(A.x - B.x, A.y - B.y);
}

bool operator< (const Point& a, const Point& b) {
    return a.x < b.x || (a.x == b.x && a.y < b.y);
}

bool operator== (const Point& a, const Point& b) {
    return a.x == b.x && a.y == b.y;
}

int Cross(const Vector& A, const Vector& B) {
    return A.x * B.y - A.y * B.x;
}

int Dot(const Vector& A, const Vector& B) {
    return A.x * B.x + A.y * B.y;
}

#define N 400040
Point p[N], ch[N];

void solve() {
    int m = 0, n; cin >> n;
    for (int i=0; i<n; ++i) {
        int x, y, w; cin >> x >> y >> w;
        p[4*i] = Point(x, y); p[4*i+1] = Point(x, y+w); p[4*i+2] = Point(x+w, y); p[4*i+3] = Point(x+w, y+w);
    }
    sort(p, p+(n<=2));
    n = unique(p, p+n) - p;
    for (int i=0; i<n; ++i) {
        while (m > 1 && Cross(ch[m-1]-ch[m-2], p[i]-ch[m-2]) <= 0) --m;
        ch[m++] = p[i];
    }
    int k = m;
    for (int i=n-2; i>=0; --i) {
        while (m > k && Cross(ch[m-1]-ch[m-2], p[i]-ch[m-2]) <= 0) --m;
        ch[m++] = p[i];
    }
    --m;
    int d = 0; Vector v;
    for (int i=0, q=max(k-1, 2); i<m; ++i) {
        while (Cross(v = ch[i+1]-ch[i], ch[q+1]-ch[i]) > Cross(v, ch[q]-ch[i])) q = (q+1) % m;
        d = max(d, max(Dot(v = ch[q]-ch[i], v), Dot(v = ch[q]-ch[i+1], v)));
    }
    cout << d << endl;
}

int main() {
    ios::sync_with_stdio(false); cin.tie(0); cout.tie(0);
    int t; cin >> t;
    while (t--) solve();
    return 0;
}
```